```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import xgboost as xgb
import shap
from sklearn.metrics import (
    classification_report, confusion_matrix,
    roc_auc_score, roc_curve,
    precision_recall_curve, f1_score
)

%matplotlib inline

train = pd.read_csv('../data/processed/training_xgb_rf.csv')
test  = pd.read_csv('../data/processed/test_xgb_rf.csv')

X_train = train.drop(columns=['label'])
y_train = train['label']
X_test  = test.drop(columns=['label'])
y_test  = test['label']

print(f'Train: {X_train.shape}, Test: {X_test.shape}')
print(f'Class balance (train) — 0: {(y_train==0).sum()}, 1:
{(y_train==1).sum()}')
```

```
Train: (107740, 58), Test: (55945, 58)
Class balance (train) — 0: 51890, 1: 55850
```

```
scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()

base_model = xgb.XGBClassifier(
    scale_pos_weight=scale_pos_weight,
    eval_metric='logloss',
    random_state=42,
    n_jobs=-1
)
```

Hyperparameter Tuning

```
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    'n_estimators':     [200, 300, 500, 700, 1000],
    'max_depth':        [3, 4, 5, 6, 8],
    'learning_rate':    [0.01, 0.05, 0.1, 0.2],
    'subsample':        [0.6, 0.7, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.7, 0.8, 1.0],
    'min_child_weight': [1, 3, 5],
    'gamma':            [0, 0.1, 0.3, 0.5],
```

```
    'reg_alpha':        [0, 0.1, 0.5, 1.0],
    'reg_lambda':       [0.5, 1.0, 2.0, 5.0],
}

search = RandomizedSearchCV(
    base_model,
    param_distributions=param_dist,
    n_iter=50,
    scoring='roc_auc',
    cv=3,
    n_jobs=-1,
    random_state=42,
    verbose=1
)

search.fit(X_train, y_train)

print(f'Best ROC-AUC (CV): {search.best_score_:.4f}')
print(f'Best params:       {search.best_params_}')
```

```
Fitting 3 folds for each of 50 candidates, totalling 150 fits
Best ROC-AUC (CV): 0.9601
Best params:       {'subsample': 0.7, 'reg_lambda': 2.0, 'reg_alpha':
1.0, 'n_estimators': 1000, 'min_child_weight': 1, 'max_depth': 4,
'learning_rate': 0.01, 'gamma': 0.1, 'colsample_bytree': 0.8}
```

```
model = search.best_estimator_

model.fit(
    X_train, y_train,
    eval_set=[(X_test, y_test)],
    verbose=50
)
```

```
[0]   validation_0-logloss:0.68752
[50]  validation_0-logloss:0.51832
[100] validation_0-logloss:0.44178
[150] validation_0-logloss:0.40293
[200] validation_0-logloss:0.38062
[250] validation_0-logloss:0.36531
[300] validation_0-logloss:0.35499
[350] validation_0-logloss:0.34740
[400] validation_0-logloss:0.34062
[450] validation_0-logloss:0.33480
[500] validation_0-logloss:0.33025
[550] validation_0-logloss:0.32661
[600] validation_0-logloss:0.32327
[650] validation_0-logloss:0.31967
[700] validation_0-logloss:0.31660
[750] validation_0-logloss:0.31311
```

```
[800] validation_0-logloss:0.31062
[850] validation_0-logloss:0.30855
[900] validation_0-logloss:0.30733
[950] validation_0-logloss:0.30610
[999] validation_0-logloss:0.30481

XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=0.8, device=None,
early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=0.1,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.01,
max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=4, max_leaves=None,
              min_child_weight=1, missing=nan,
monotone_constraints=None,
              multi_strategy=None, n_estimators=1000, n_jobs=-1,
              num_parallel_tree=None, ...)
```

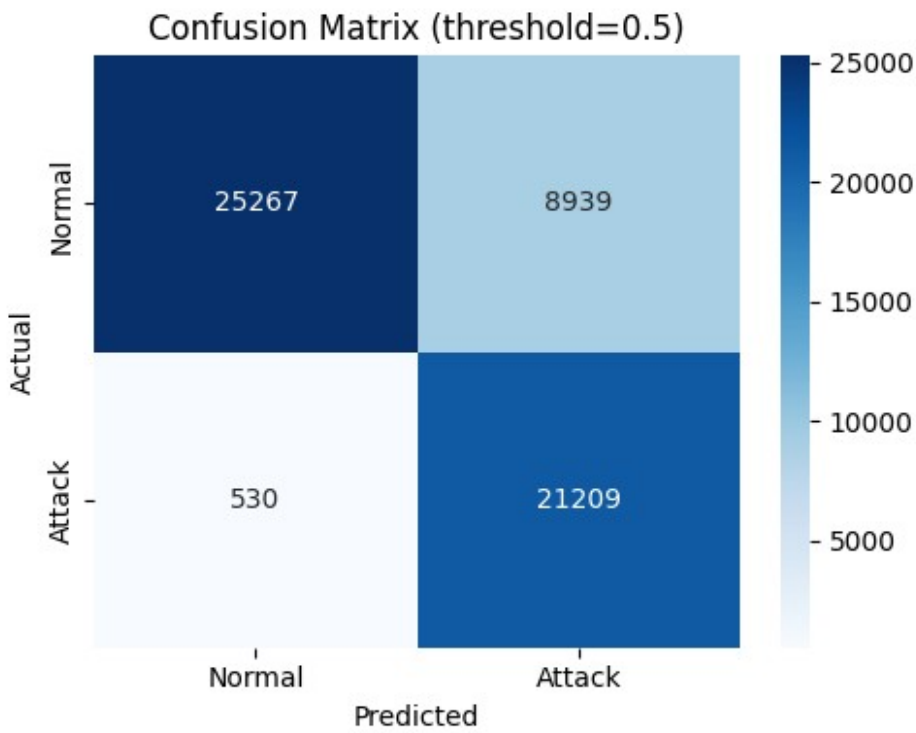
Base Evaluation, threshold = 0.5

```
y_prob = model.predict_proba(X_test)[:, 1]
y_pred = (y_prob >= 0.5).astype(int)

print(classification_report(y_test, y_pred, target_names=['Normal',
'Attack']))
print(f'ROC-AUC: {roc_auc_score(y_test, y_prob):.4f}')
```

```
              precision    recall  f1-score   support

      Normal       0.98      0.74      0.84     34206
      Attack       0.70      0.98      0.82     21739

    accuracy                           0.83     55945
   macro avg       0.84      0.86      0.83     55945
weighted avg       0.87      0.83      0.83     55945

ROC-AUC: 0.9695
```

```
cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
            xticklabels=['Normal', 'Attack'], yticklabels=['Normal',
'Attack'])
ax.set_ylabel('Actual')
ax.set_xlabel('Predicted')
ax.set_title('Confusion Matrix (threshold=0.5)')
```

```
plt.tight_layout()
plt.show()
```



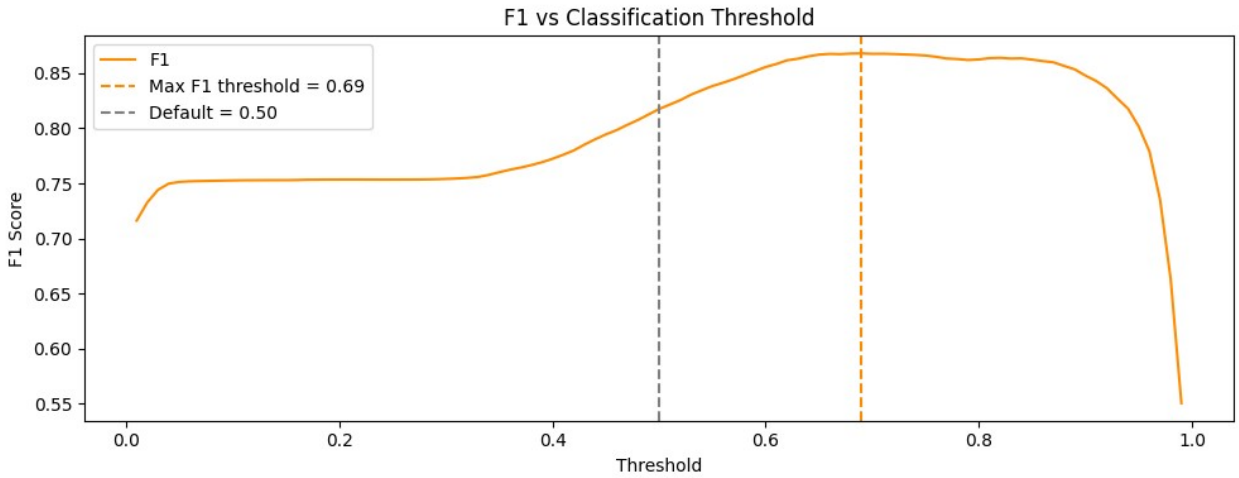
Threshold Tuning

In cybersecurity, false negatives (missed attacks) are more costly than false positives. We find the threshold that maximises F1, then also examine the recall-precision tradeoff.

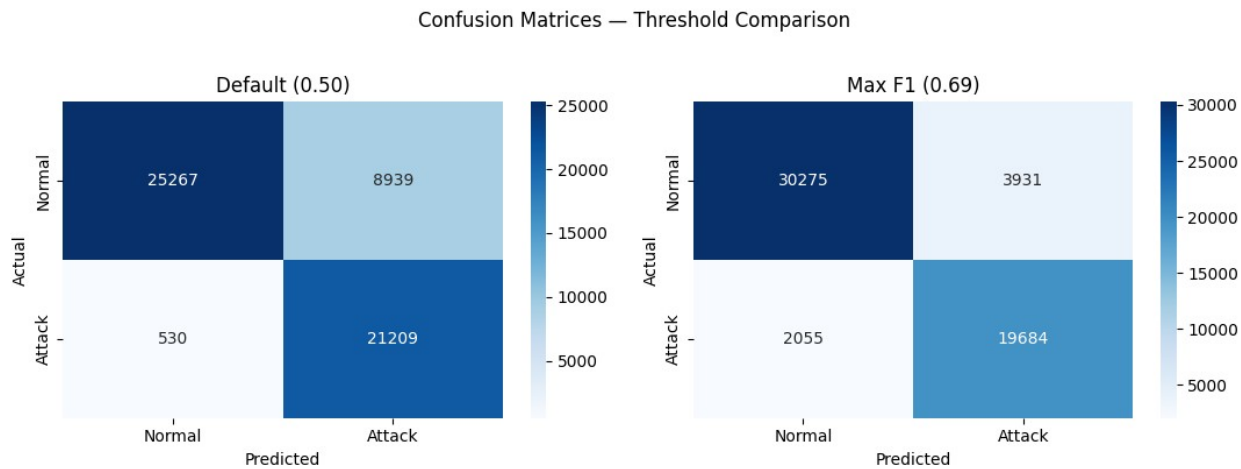
```
from sklearn.metrics import recall_score

thresholds = np.arange(0.01, 1.0, 0.01)
f1_scores  = [f1_score(y_test,    (y_prob >= t).astype(int)) for t in
thresholds]

best_f1_threshold = thresholds[np.argmax(f1_scores)]

print(f'Optimal threshold (max F1): {best_f1_threshold:.2f}  — F1:
{max(f1_scores):.4f}')
print(f'Default threshold (0.5):           — F1: {f1_scores[49]:.4f}')

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(thresholds, f1_scores, color='darkorange', label='F1')
ax.axvline(best_f1_threshold, color='darkorange', linestyle='--',
label=f'Max F1 threshold = {best_f1_threshold:.2f}')
ax.axvline(0.5, color='gray', linestyle='--', label='Default = 0.50')
ax.set_xlabel('Threshold')
```

```
ax.set_ylabel('F1 Score')
ax.set_title('F1 vs Classification Threshold')
ax.legend()
plt.tight_layout()
plt.show()
```

```
Optimal threshold (max F1): 0.69  — F1: 0.8680
Default threshold (0.5):           — F1: 0.8175
```




```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
for ax, (t, title) in zip(axes, [
    (0.5,                 'Default (0.50)'),
    (best_f1_threshold, f'Max F1 ({best_f1_threshold:.2f})'),
]):
    preds = (y_prob >= t).astype(int)
    sns.heatmap(confusion_matrix(y_test, preds), annot=True, fmt='d',
cmap='Blues', ax=ax,
                xticklabels=['Normal', 'Attack'],
yticklabels=['Normal', 'Attack'])
    ax.set_ylabel('Actual')
    ax.set_xlabel('Predicted')
    ax.set_title(title)
plt.suptitle('Confusion Matrices — Threshold Comparison', y=1.02)
plt.tight_layout()
plt.show()

print(f'\nChosen threshold: {best_f1_threshold:.2f} (max F1)')
print(classification_report(y_test, (y_prob >=
best_f1_threshold).astype(int), target_names=['Normal', 'Attack']))
```




```
Chosen threshold: 0.69 (max F1)
              precision    recall  f1-score   support

      Normal       0.94      0.89      0.91     34206
      Attack       0.83      0.91      0.87     21739

    accuracy                           0.89     55945
   macro avg       0.88      0.90      0.89     55945
weighted avg       0.90      0.89      0.89     55945
```

```
y_pred_tuned = (y_prob >= best_f1_threshold).astype(int)
```

Per-Attack-Category Breakdown

Join predictions against the raw test data to evaluate performance per attack type.

```
raw_test = pd.read_csv('../data/raw/UNSW_NB15_testing-set.csv')

raw_test = raw_test.drop_duplicates(subset=[c for c in
raw_test.columns if c != 'id']).reset_index(drop=True)

results = raw_test[['attack_cat']].copy()
results['y_true']       = y_test.values
results['y_pred']       = y_pred_tuned
results['y_prob']       = y_prob

results.head()
```

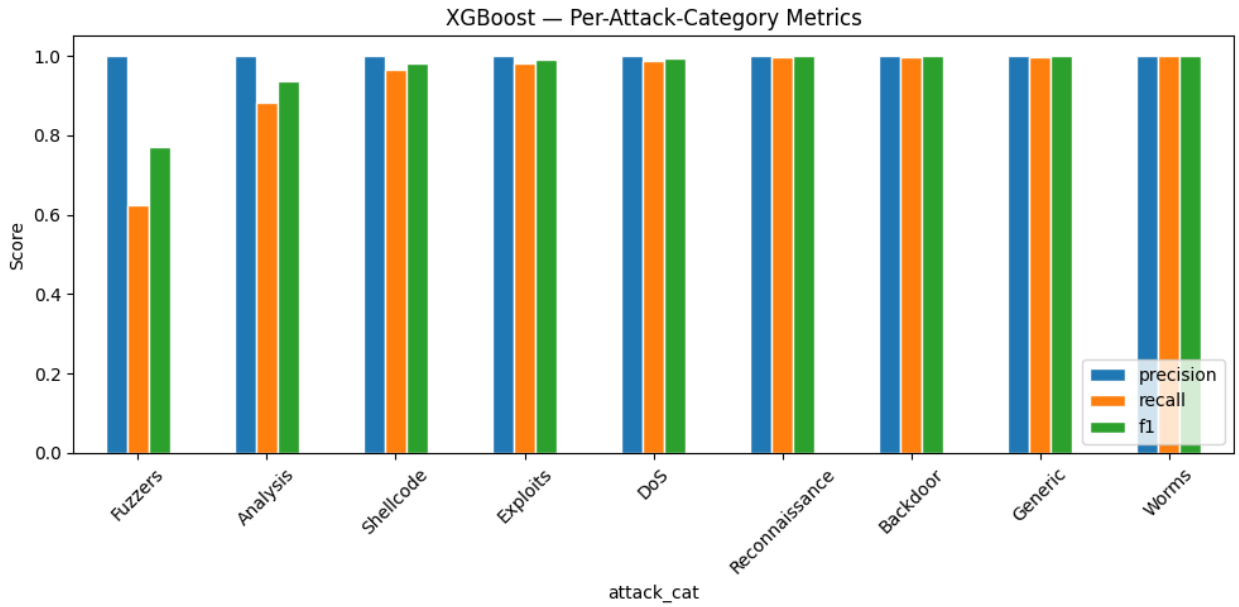
```
  attack_cat  y_true  y_pred    y_prob
0     Normal       0       1  0.862386
1     Normal       0       1  0.804883
2     Normal       0       1  0.702320
```

```
3     Normal       0       1  0.707702
4     Normal       0       1  0.709230
```

```
def attack_metrics(group):
    tp = ((group['y_true'] == 1) & (group['y_pred'] == 1)).sum()
    fn = ((group['y_true'] == 1) & (group['y_pred'] == 0)).sum()
    fp = ((group['y_true'] == 0) & (group['y_pred'] == 1)).sum()
    tn = ((group['y_true'] == 0) & (group['y_pred'] == 0)).sum()
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall    = tp / (tp + fn) if (tp + fn) > 0 else 0
    f1        = 2 * precision * recall / (precision + recall) if
(precision + recall) > 0 else 0
    return pd.Series({'count': len(group), 'precision':
round(precision, 4),
                      'recall': round(recall, 4), 'f1': round(f1, 4)})

attack_results = results[results['attack_cat'] != 'Normal']
breakdown = attack_results.groupby('attack_cat').apply(attack_metrics,
include_groups=False).sort_values('recall')
print(breakdown.to_string())
```

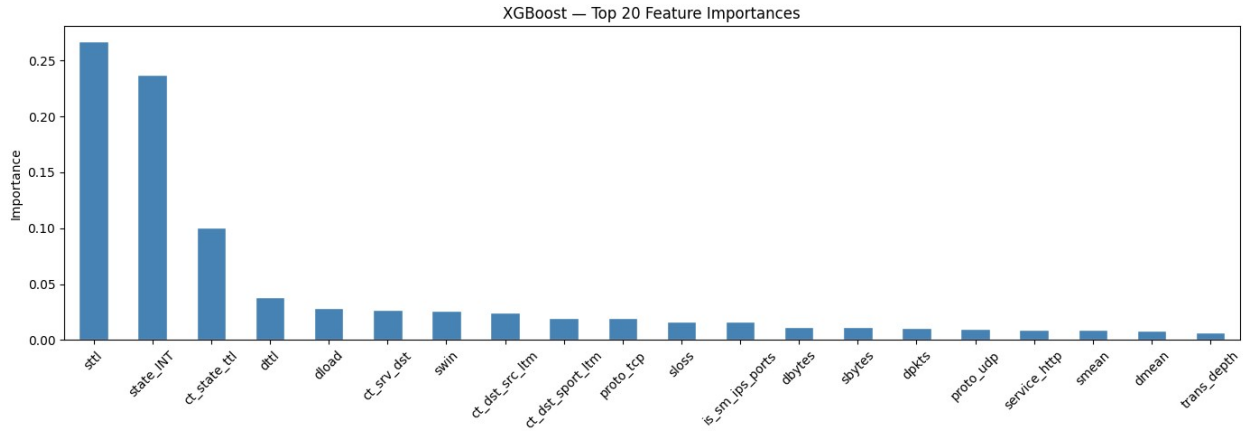
```
                  count  precision  recall      f1
attack_cat
Fuzzers          4838.0        1.0  0.6255  0.7696
Analysis          446.0        1.0  0.8812  0.9368
Shellcode         378.0        1.0  0.9656  0.9825
Exploits         7609.0        1.0  0.9823  0.9910
DoS              1718.0        1.0  0.9866  0.9933
Reconnaissance   2703.0        1.0  0.9970  0.9985
Backdoor          346.0        1.0  0.9971  0.9986
Generic          3657.0        1.0  0.9973  0.9986
Worms              44.0        1.0  1.0000  1.0000
```

```
fig, ax = plt.subplots(figsize=(10, 5))
breakdown[['precision', 'recall', 'f1']].plot(kind='bar', ax=ax,
edgecolor='white')
ax.set_title('XGBoost — Per-Attack-Category Metrics')
ax.set_ylabel('Score')
ax.set_ylim(0, 1.05)
ax.tick_params(axis='x', rotation=45)
ax.legend(loc='lower right')
plt.tight_layout()
plt.show()
```

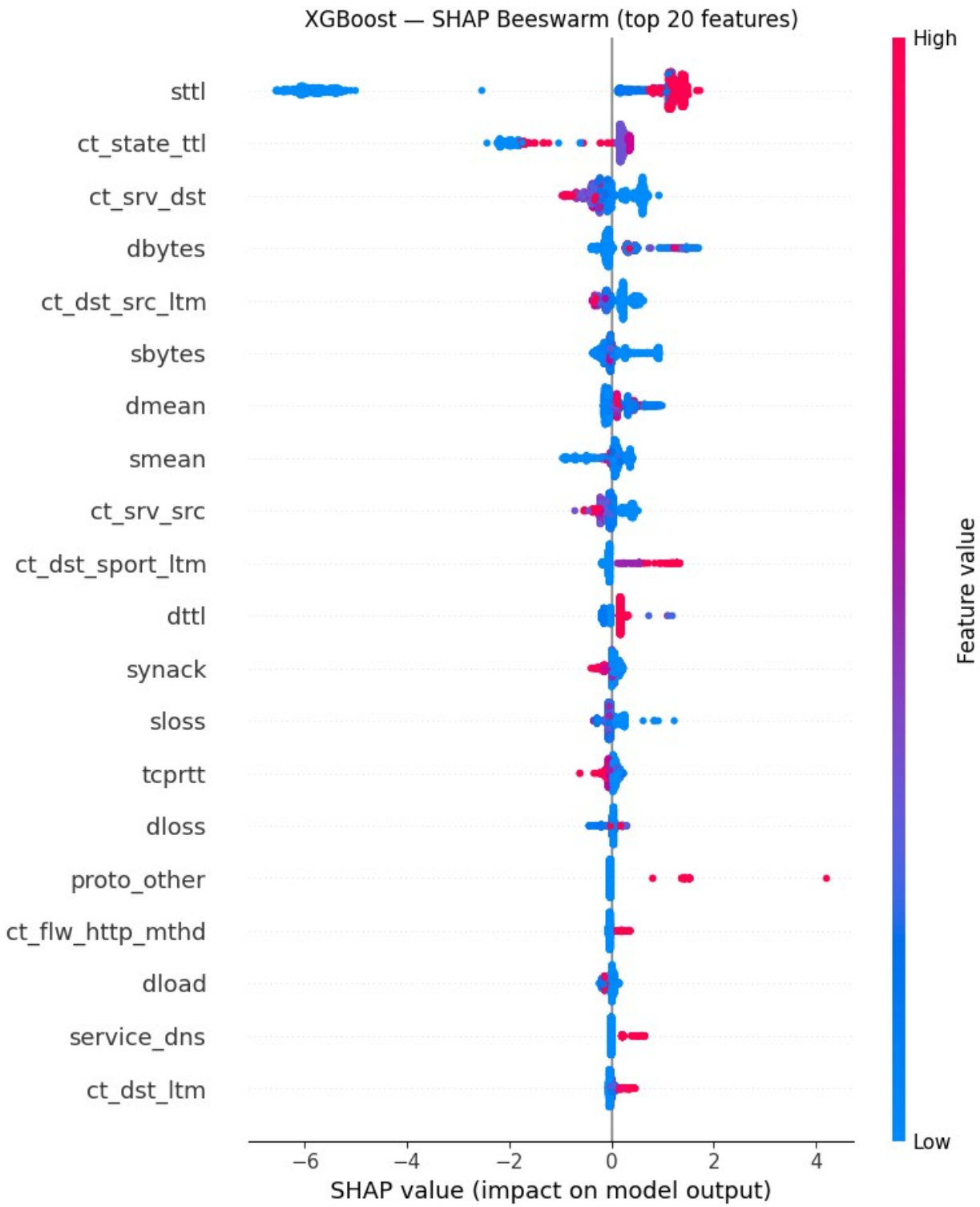


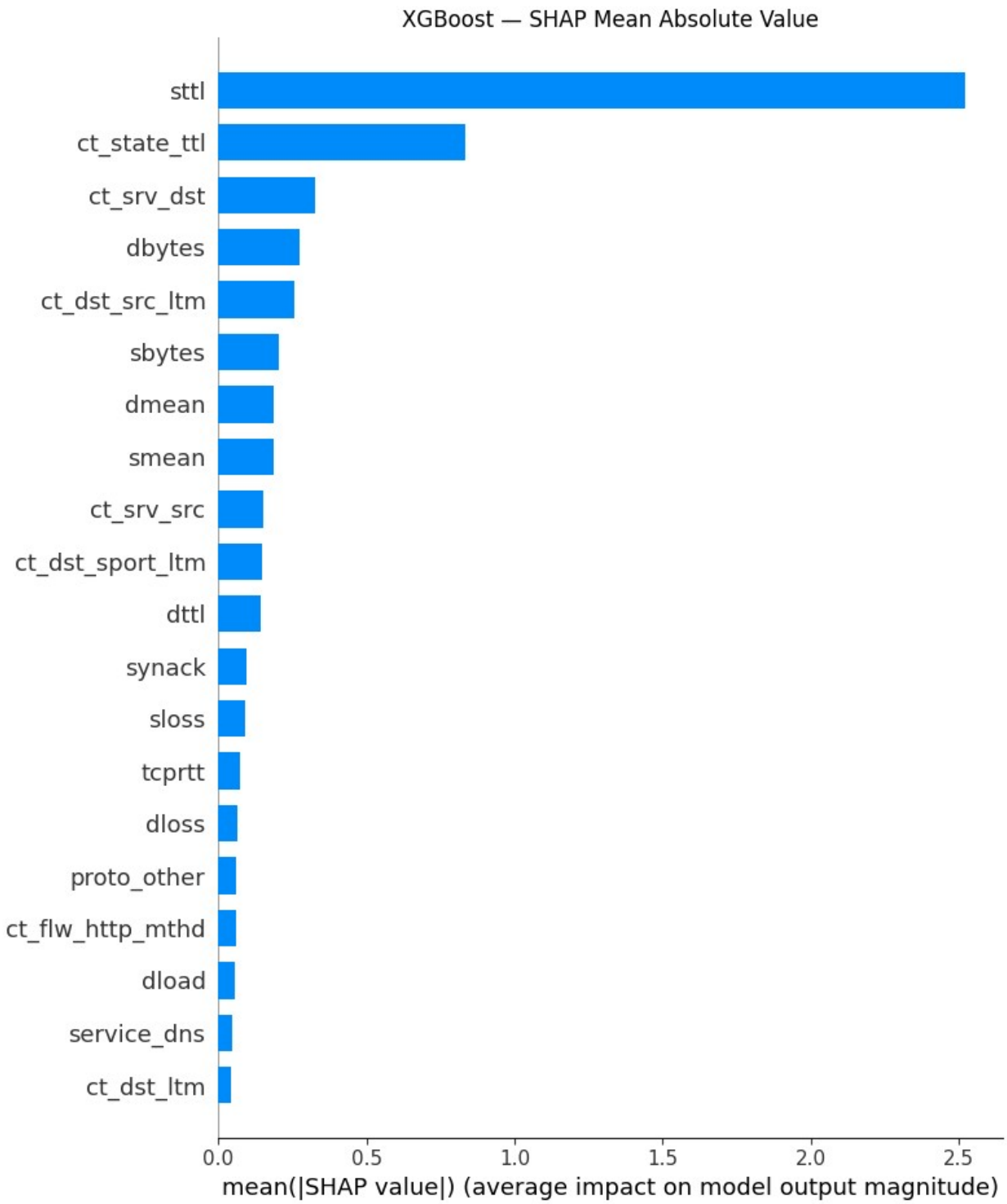
Feature Importance

```
importances = pd.Series(model.feature_importances_,
index=X_train.columns).sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(14, 5))
importances.head(20).plot(kind='bar', ax=ax, color='steelblue',
edgecolor='white')
ax.set_title('XGBoost — Top 20 Feature Importances')
ax.set_ylabel('Importance')
ax.tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.show()
```



```python
X_sample = X_test.sample(n=2000, random_state=42)

explainer   = shap.TreeExplainer(model)
shap_values = explainer(X_sample)

# Beeswarm — magnitude + direction per feature
shap.summary_plot(shap_values, X_sample, max_display=20, show=False)
plt.title('XGBoost — SHAP Beeswarm (top 20 features)')
plt.tight_layout()
plt.show()

# Bar — mean absolute SHAP (global ranking)
shap.summary_plot(shap_values, X_sample, plot_type='bar',
max_display=20, show=False)
plt.title('XGBoost — SHAP Mean Absolute Value')
plt.tight_layout()
plt.show()
```


XGBoost — SHAP Beeswarm (top 20 features)
sttl
ct_state_ttl
ct_srv_dst
dbytes
ct_dst_src_ltm
sbytes
dmean
smean
ct_srv_src
ct_dst_sport_ltm
dttl
synack
sloss
tcprtt
dloss
proto_other
ct_flw_http_mthd
dload
service_dns
ct_dst_ltm
−6
−4
−2
0
2
4
SHAP value (impact on model output)
High
Feature value
Low


XGBoost — SHAP Mean Absolute Value
sttl
ct_state_ttl
ct_srv_dst
dbytes
ct_dst_src_ltm
sbytes
dmean
smean
ct_srv_src
ct_dst_sport_ltm
dttl
synack
sloss
tcprtt
dloss
proto_other
ct_flw_http_mthd
dload
service_dns
ct_dst_ltm
0.0
0.5
1.0
1.5
2.0
2.5
mean(|SHAP value|) (average impact on model output magnitude)